École d'ingénieur Denis Diderot
Université Paris Cité

THALES

Implementation of the language server protocol and a Visual Studio Code extension for C/ACSL programming language features.

July 30, 2024

Internship report

Djamila Mohamed
2023–2024

Supervisor:
Adel Djoudi, Formal Methods Expert

Contents

Part I
Introduction

1 Thales DIS, IBS

Thales Digital Identity and Security is the global business unit (GBU) of Thales which operates in the field of secure software, providing biometric and encryption solutions. Within the DIS GBU is the Identity and Biometric Solutions business line (BL) which is in charge of digital identity (e.g.: cards and passports). The IBS BL is divided in three subunits: ID Documents Solutions (IDDS), Digital ID & Verification Solutions and Public Security. I am assigned to the IDDS subunit as an intern software developer, under the supervision of Adel Djoudi, formal methods expert at Thales DIS.

2 Context

In IDDS, developers use program analyzers such as Frama-C[1], a C source file analyzer developed by CEA and Inria. This software provides a precise analysis of code and potential security flaws by expressing functional specifications through ANSI/ISO C Specification Language (ACSL)[2] annotations. But Frama-C currently does not provide a convenient integrated development environment (IDE) to edit and browse C and ACSL code.

Writing ACSL annotations for C functions requires browsing the Frama-C libraries and search for all the needed functions in it. Although this method works for those familiar with Frama-C, it consumes time and there is a need for a quicker way to access the definitions and declarations of these functions.

Additionally, those unaccustomed to Frama-C and ACSL annotations can benefit from this solution for getting started and have a better understanding of the specification language.

The goal of this project is to develop a software solution to offer language programming features for C/ACSL in IDEs including code navigation, completion suggestions, and interaction with Frama-C code analysis.

Part II
Background

3 The Language Server Protocol

3.1 Introduction

Developers working on large scale projects often need a code editing environment with the adequate tooling for the languages they use in their workspace. Multiple solutions such as Eclipse or IntelliJ IDEA exist for Java projects, as well as other popular programming languages. But the complexity of developing one tool for one language and one specific editor is tedious as it forces developers to implement the same language programming features for every code editor with each programming language. To overcome these, a recent solution was developed: the Language Server Protocol (LSP)[3].

The language Server Protocol (LSP) is built on top of JSON-RPC protocol, a remote procedure call protocol that uses JSON as data format.

3.2 LSP Capabilities

The LSP defines sets of features named ‘capabilities’. LSP capabilities are boolean values transported in a JSON requests that each party declares when the extension is launched. These declarations are needed so that both agree on which features each can provide. Only the features supported by both parties can be used in the editor. Capabilities are defined as TypeScript interfaces named ‘ClientCapabilities’ and ‘ServerCapabilities’ in the protocol specification.

Figure 1: Sequence diagram of the ‘initialize’ handshake

[1] https://frama-c.com/html/overview.html
[2] https://frama-c.com/html/acsl.html
[3] https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/

3.3 Use cases for each LSP feature

4 Frama-C plugin development

We choose to implement the ACSL language server as a Frama-C plugin to leverage the parsing features of Frama-C for ACSL code. It was therefore necessary to read and follow the Frama-C (Nickel 28.0) plug-in development guide. The Frama-C plug-in will act as the server for the LSP and the VS Code extension as the client. The plug-in development is the main task of the project. VS Code was chosen since it is used by most developers especially in Thales DIS. As long as the server is implemented, any editor can provide an extension to connect to the Frama-C server.

5 VS Code Extensions

The architecture of VS Code extensions is based on a client-server model, which aligns perfectly with the Language Server Protocol. VS Code provides an API with TypeScript libraries for developing different types of extensions, particularly, language extensions. A language extension offers declarative (e.g. syntax highlighting, bracket autoclosing) and programmatic (e.g. hover information, jump to definition) language features.

Part III
Installation

6 Dependencies

6.1 Run

6.2 Debug

To perform these instructions you will need the following dependencies:

- node >= 18.18.0
- npm >= 9.8.0 (npm is installed if you have node installed)
- vsce
- opam
- dune
- frama-c 28.0

7 Frama-C plug-in

1. Open terminal and clone git project 'Acsl_lsp'
2. Change opam switch : 'opam switch 4.13.1_fc28'
3. In the 'server' folder, execute 'dune build' then 'dune install'

8 VS Code extension

If you need to compile, perform these instructions before (you can skip this part if you are not debugging):

1. In the project cloned previously, go to 'client'
2. Run 'nvm use 18.18.0' if you have it installed else update it first with 'node update 18.18.0', then run 'npm install', 'npm run compile'
3. Run 'vsce package' in the same folder, this will generate a v̇six file

Else:

1. Go to the 'Extensions' view and click on the three dots 'Views and more actions...'

2. Select ‘Install from VSIX...’
3. Give the path to the v̇six file located in the ‘client’ folder

Part IV
Development

9 Server implementation

One of the primary tasks is to select the appropriate method for communicating between the VS Code client and the Language Server. In this case, the implementation requires leveraging the OCaml Unix module and using the TCP socket transport method to establish a connection between the two components. This communication channel enables the exchange of requests and responses.

To facilitate the communication between client and server, the extension needs to handle both receiving and sending requests using the Unix module in OCaml. The VS Code client creates a server process listening on ‘localhost:8001’, the Frama-C plug-in will connect to that address and receive requests from it. Port 8001 was chosen arbitrarily for development purposes.

When client and server are connected, they exchange their capabilities through the ‘initialize’ handshake. The VS Code process is executed in a Node.js runtime environment.

Below is a sequence diagram representing the lifecycle of the VS Code application.

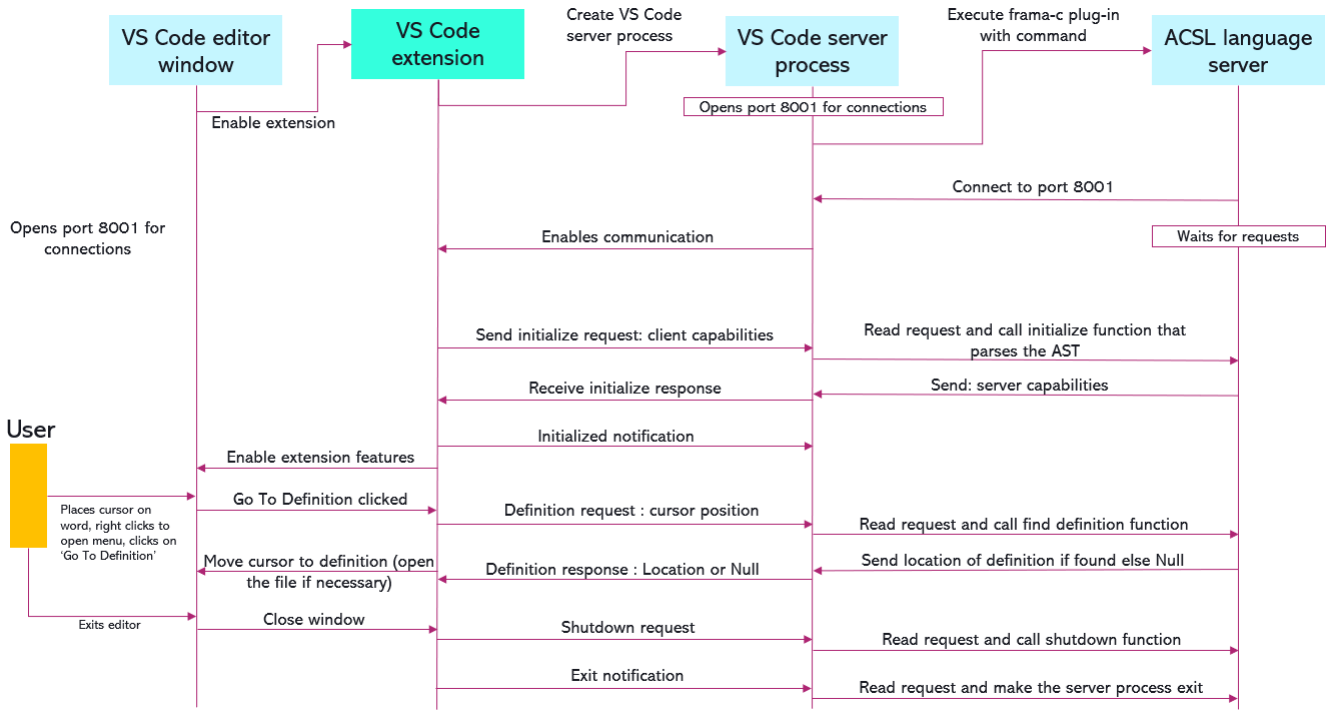

Figure 2: Lifecycle of the VS Code language extension for ACSL

10 File parsing, put this at the end

In order to use the Frama-C module’s functions, the plug-in converts the source code files into their C Intermediate Language (CIL) representation, which provides a detailed hierarchical structure of the source code. It captures the syntactic elements and relationships within the code, for deeper analysis and processing. This representation allows the server to efficiently navigate and manipulate the code, supporting features like ‘Go To Definition’ and other code navigation tools.

The system is designed to handle the parsing of both single and multiple source code files. Regardless of the number of files being parsed, the resulting Abstract Syntax Trees (ASTs) from each file are merged into a single one. This merging process is crucial as it simplifies the analysis and manipulation of the code, providing a unified structure that the server can work with more effectively.

When dealing with large codebases, users often face the challenge of working with a vast number of source files. Parsing all these files can be a time-consuming task. To address this issue, the system offers a Visual Studio Code setting that allows users to specify which source files need to be parsed. By focusing the parsing process on only the relevant files, this setting helps optimizing the time spent on parsing, making the workflow more efficient.

Users must ensure that all necessary source files are included in the parsing process. If a user fails to specify a source file that is needed for certain features, such as Go To Definition, they may not function correctly. Therefore, it is the responsibility of the user to make sure they provide all the relevant files.

11 'Go To Definition' feature

The 'Go To Definition' feature of Visual Studio Code allows developers to easily navigate to the definition of symbols within their code. In this context, the challenges involve implementing a VS Code extension that supports this feature for two languages associated with the same file extension ('.c' and '.h').

When a user requests to find the definition of a symbol at a specific position in the code, the server first identifies the name of the symbol located at that position. This involves parsing, analyzing the file and extracting the relevant word based on the provided line and column numbers. The code then initiates a search through various sections of the code to find where the symbol is defined. This search involves visiting different parts of the source file's Abstract Syntax Tree (AST) and concerns both ACSL and C code:

- ACSL Definitions: The code checks if the symbol matches any logical predicates or terms, which are specific to tools like Frama-C used for static analysis of C programs.
- C Definitions: It examines global entities, such as enums, structures, functions, and variables, to see if the symbol matches any of these.

Each of these searches uses visitor patterns, which traverse and inspect nodes in the AST, comparing each node's name to the symbol's name.

If a definition is located, the exact file path and the range of lines and columns where the symbol is defined are packaged into a JSON response. If no definition is found, a response indicating a null result is returned. This response allows the LSP client (the user's text editor or IDE) to navigate directly to the symbol's definition or handle the case where no definition exists.

12 File Synchronization & Diagnostics

File synchronization is a key (and mandatory) feature of the LSP. Three types of requests are sent by VS Code to notify document opening, changing and saving actions performed by the user. A single file parsing is performed each time a file is opened or saved. We add an event listener to catch any error or warning raised by the Frama-C parser and we store them inside a JSON structure specifically designed by the LSP for the 'Diagnostics' feature. All diagnostics are published to the editor and displayed to the user in the form of squiggles and listed in the 'Problems' section of the VS Code terminal/console (?).

There are two types of diagnostics: warnings and errors. Warnings are raised in case an ACSL annotation is not formed properly or when an unknown function is called. Errors are raised when there is a syntax error in the C code preventing the parser to finish (which are fatal errors). In the latter case, normally, any Frama-C session stops, but we ensure that diagnostics are sent either in case the parsing succeeds or fails. So we send them just before the exception aborts fatally. In case the parsing succeeds without any messages, the diagnostics are cleared in the editor.

The listener is a Frama-C function that reports any event emitted by Frama-C while parsing. From those events we can extract their source, their message and their kind (whether it is an error, a warning a feedback, etc.). By manipulating these data, we organize them and adapt them to the LSP standard.

This feature is only implemented for single file parsing and diagnostics are computed only when the file is saved. No diagnostics are published for multiple file parsing, which means we do not know if those files are in state of error. So the user has to make sure the source files they provided have no errors.

Part V
Ongoing tasks

- Create end-to-end tests for VS Code extension
- Create tests for Frama-C plug-in
- Implement 'Diagnostics' feature of LSP (compilation errors, warnings, etc.)

- Implement the display of CIL of source code on demand